Actividad 6. Implementación y uso del TAD Map

Objetivo

Conocer el TAD MAP como estructura adecuada para almacenar una colección de objetos de la forma clave/valor, aprender a implementar dicho TAD y conocer la utilidad del TAD Map para resolver problemas.

Procedimiento

Para resolver esta actividad se seguirá el siguiente procedimiento:

1. Formar equipos de 4 personas del mismo grupo reducido, que trabajarán juntos durante 3 semanas.
2. En la primera semana de trabajo todo el equipo debe formarse en el diseño del TAD Map. A continuación, cada equipo se dividirá en 2 subequipos (de 2 personas cada uno). Cada subequipo resolverá un ejercicio de esta actividad. En esta semana se resolverán los ejercicios 1 y 2a.
3. En la segunda semana, los subequipos intercambiarán su composición y, de nuevo por pares, resolverán los ejercicios 2b y 2c.
4. En la tercera semana, se garantizará que todos los ejercicios están resueltos. Entonces, las 4 personas del equipo se reunirán y habrá una puesta en común de los ejercicios realizados, de forma que todos conozcan perfectamente cómo se resolvieron todos los ejercicios propuestos en esta actividad.
5. Si el equipo de trabajo no funciona o si alguien del equipo no es responsable de su trabajo, debe informarse al/a la docente.

Evaluación

Los contenidos de esta actividad serán evaluados mediante una prueba individual el día 19 de noviembre de 2025.

Tiempo estimado

7 horas

Ejercicios para clase

1.- Dada la especificación del TAD Map<K,V> que se muestra a continuación

TAD MAP<K,V>

Características: almacena una colección de objetos <clave, valor>. No contiene claves repetidas. No permite claves null. Los objetos son modificables

```
public Map()
```
//*Produce*: crea un map vacío

```
public int size();
```
//*Produce*: devuelve el número de pares <clave, valor> del map

```
public V get (K key);
```
//*Produce*: devuelve el valor asociado a la clave que se pasa como parámetro o null cuando la clave no existe

```
public void add (K key, V value) throws NullPointerException;
```
//*Modifica*: this
//*Produce*: Si clave es null lanza NullPointerException; en otro caso inserta el par <clave, valor> en el map. Si la clave ya existe en el map, reemplaza el valor viejo por el nuevo valor que se pasa como parámetro.

```
public V remove (K key);
```
//*Modifica*: this
//*Produce*: elimina la clave del map y su valor asociado, el cual se retorna. Si la clave no existe devuelve null.

```
public Set<K> getKeys();
```
//*Produce*: devuelve un conjunto con las claves del map

```
public Iterator<V> getValues();
```
//*Produce*: devuelve un iterator sobre los valores del map, de forma que el iterador devuelto accede directamente a la estructura interna del mapa, sin realizar copias. Esto implica que las modificaciones a través del iterador afectan directamente al estado del mapa.

```
public void clear();
```
//*Produce*: elimina todos los elementos del map, convirtiéndolo en un map vacío.

Se pide la implementación del TAD Map<K,V>, es decir, escribir una clase que implemente la interfaz Map<K,V> definida previamente, haciendo uso de una Tabla Hash (HashMap<K,V> implements Map<K,V>). Para ello haz uso de la clase `HashMap` situada en el paquete `es.uvigo.esei.aed2.activity6.HashMap`, la cual contiene las cabeceras de los métodos.

La **Tabla Hash** se utiliza para almacenar una colección de objetos <clave, valor>. Para resolver las colisiones que se produzcan cuando se realicen inserciones en la Tabla Hash, ésta se va a implementar siguiendo la estrategia de *encadenamiento abierto o separado* (un array de listas). Para trabajar con listas se recomienda usar la interface *List<E>* de java.

Durante el proceso de búsqueda, inserción o borrado de un elemento de la Tabla Hash, el primer paso a realizar es transformar la clave en un índice del array, es decir, aplicar una función hash. Este índice indicará la lista en la que hay que buscar, insertar o eliminar el par clave/valor correspondiente. Esta función debe implementarse mediante un método privado que, para esta actividad, codificará el *método de división*:

```
private int calculateIndex (K key)
```

Un problema de esta función (método de división) es que para poder aplicarla es necesario que el objeto que se pasa como parámetro sea de tipo *int*. Para resolver este problema existe en java el método **hashCode()**, que convierte cualquier objeto en un *int*. Es decir, **key.hashCode()** convierte el parámetro de entrada de la función a un valor entero, lo cual permite gestionar cualquier objeto en la Tabla Hash.

Para probar que la implementación realizada funciona correctamente se proporciona la clase de prueba `HashMapTestCase`.

2.- Haciendo uso del TAD Map<K,V> diseñado, resuelve los problemas que se indican a continuación:

a) Dos palabras son **anagramas** si tienen las mismas letras, con la misma frecuencia, pero en distinto orden. Ejemplos de anagramas: cara-arca, animal-lamina, saco-cosa.

Se pide escribir un método que, dada una lista de palabras y haciendo uso del TAD Map, devuelva, agrupados, los anagramas que se encuentran en dicha lista. Por ejemplo: dadas las palabras "rato", "arto", "tabo", "par", "taro", "bato", devuelva el siguiente agrupamiento de anagramas:

[

["rato", "arto", "taro"],

["par"],

["tabo", "bato"]

]

La sintaxis del método a implementar sería:

```
public static Set<Set<String>> getAnagrams(List<String> words)
```

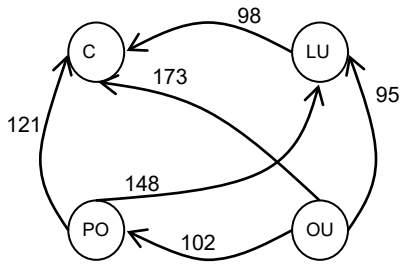
Para resolver el problema se aconseja:

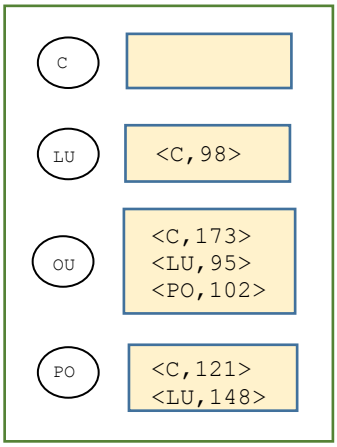
- Usar como **clave** del Map<K,V> una representación única de las palabras anagramas. Lo más sencillo es ordenar las letras de las palabras alfabéticamente. Es decir, las palabras "rato", "arto" y "taro" tendrían como clave "aort"
- Usar como **valor** del Map<K,V> la lista de palabras que son anagramas entre sí.

Para llevar a cabo la implementación, completa la clase `Anagrams` del paquete es.uvigo.esei.aed2.activity6.anagrams. Una vez realizada la implementación, prueba su correcto funcionamiento con la clase de test `AnagramsTestCase`.

b) Una implementación eficiente del TAD Graph<T,E> es posible mediante mapas de adyacencia Map<K,V>. Es una implementación en la que los vértices del grafo se guardan como claves, Map<Vertex<T>, V>, y los adyacentes a cada vértice son los valores, los cuales se guardan también un TAD Map<K,V>. Es decir, los vértices adyacentes se guardan en un Map<Vertex<T>,E> donde el vértice adyacente actúa como clave y la etiqueta del arco actúa como valor.

Ejemplo:





Haciendo uso, exclusivamente, de tal representación interna:

```
public class MapOfMap<T,E> implements Graph<T,E>{
   private Map<Vertex<T>, Map<Vertex<T>, E>> mapOfVertices;
```

se pide implementar los métodos de la interfaz Graph<T,E>. Para ello completa la clase `MapOfMap` que está en el paquete `es.uvigo.esei.aed2.activity6.mapofmap`:

```
public interface Graph<T,E>{
    public boolean isEmpty ();
    public int numberOfVertices();
    public boolean containsVertex(Vertex<T> vertex);
    public boolean containsEdge(Vertex<T> source, Vertex<T> target, E label);
    public Set<Vertex<T>> getVertices();
    public Set<Edge<T, E>> getEdges();
    public Set<Vertex<T>> getAdjacentsVertex(Vertex<T> vertex) throws NullPointerException;
    public boolean addVertex(Vertex<T> vertex) throws NullPointerException;
    public boolean addEdge(Vertex<T> source, Vertex<T> target, E label) throws
                                                    NullPointerException, IllegalArgumentException;

    public boolean removeVertex(Vertex<T> vertex) throws NullPointerException;
    public boolean removeEdge(Vertex<T> source, Vertex<T> target, E label) throws
                                                                    NullPointerException;
    public void clear();
}
```

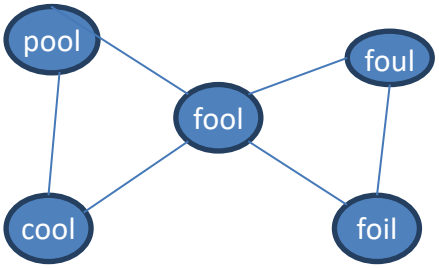
```
public class Vertex<T> {
    private T value;
    public public Vertex(T value) throws NullPointerException{}
    public T getValue(){}
    public boolean equals(Object o){}
}

public class Edge<T, E>{
    private Vertex<T> source;
    private Vertex<T> target;
    private E label;

    public Edge(Vertex<T> source, Vertex<T> target, E label) throws NullPointerException{ç
    public Vertex<T> getSource(){}
    public Vertex<T> getTarget(){}
    public E getLabel() {}
    public boolean equals(Object o) {}
}
```

Una vez realizada la implementación, prueba su correcto funcionamiento haciendo uso de la clase de test `MapOfMapTestCase`.

c) Dado un conjunto de palabras del mismo tamaño, se quiere construir un grafo que represente las relaciones entre las palabras. El grafo relaciona dos palabras si se diferencian por una sola letra. La siguiente figura muestra un pequeño grafo de algunas palabras en inglés.



Para ello se propone la siguiente solución. A partir de cada palabra del conjunto vamos a crear varias claves, cada una formada por las letras de la palabra menos una, que se sustituye por un guion. Por ejemplo, para las siguientes palabras {"pool", "cool", "fool", "foul", "foil"} podríamos tener las siguientes claves {"_ool", "p_ol", "po_l", "poo_", "c_ol", "co_l", "coo_", "f_ol", "fo_l" "foo_", ….. }

A continuación, cada palabra del conjunto inicial se guarda con su clave correspondiente, es decir, con aquella clave con la que coinciden todos los caracteres, considerando el guion un comodín. De esta forma, todas las palabras que están con la misma clave sabemos que solo se diferencian en un carácter y por lo tanto tienen que estar conectadas en el grafo. Por ejemplo:

Clave	**Lista de palabras**
_ool	pool, cool, fool

p_ol	pool
po_l	pool
…	…
fo_l	fool, foul, foil
…	

Este esquema se puede implementar haciendo uso de un **diccionario**, asociando a cada clave la lista de palabras que se guardan con ella.

Una vez creado el diccionario se puede construir el grafo fácilmente. Los vértices van a ser las palabras del conjunto inicial y las aristas se crean entre los vértices que están en la misma lista.

Escribe un programa que convierta una colección de palabras en un grafo:

```
public static Graph<String, Integer> builGraph(List<String> words)
```

Para ello completa la clase `RelatedWords` situada en el paquete `es.uvigo.esei.aed2.activity6.relatedwords`.

Una vez realizada la implementación, prueba su correcto funcionamiento con la clase de test `RelatedWordsTestCase`.

Ejercicios para casa

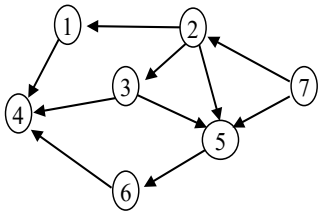
d) Ordenamiento topológico

Un grafo dirigido acíclico (DAG - Directed Acyclic Graph) es un tipo de grafo que cumple con las siguientes características: (i) Dirigido, es decir, las aristas (o conexiones entre vértices) tienen una dirección. (ii) Acíclico, es decir, no contiene ciclos. Esto significa que no puedes empezar en un vértice, seguir las aristas en su dirección, y volver al mismo nodo.

Un algoritmo habitual que se puede aplicar a un DAG es el **algoritmo de Kahn**. Este algoritmo permite obtener un ordenamiento topológico, es decir, una forma de organizar los nodos de un grafo acíclico dirigido (DAG) de tal manera que para cada arista que va de un vértice A a un vértice B, el vértice A aparece antes que B en el orden.

Un ejemplo clásico de aplicación de este algoritmo es la construcción de un orden de asignaturas a partir de parejas de prioridad. Es decir, sabiendo que para cursar la asignatura B hay que cursar antes la A, que para la D hay que cursar antes la S, etc., podemos encontrar un orden en el que matricularse de forma que cuando estudies una asignatura ya hayas cursado sus prerrequisitos.

Ejemplo:



Un posible ordenamiento topológico sería: 7, 2,1,3,5,6,4

El algoritmo de Kahn permite encontrar el ordenamiento topológico de un DAG. Su funcionamiento es el siguiente:

- Se calcula el grado de entrada de cada vértice y se guarda dicha información en un Map<Vertice<E>, Integer>.
- Se crea una cola a la que se añaden los vértices con grado 0 y se eliminan del mapa.
- Se sacan vértices de la cola hasta que queda vacía. Para cada vértice que se saca, se añade a la salida y se resta 1 al grado de los vértices adyacentes. Si el grado de alguno de estos vértices adyacentes pasa a ser 0, se añade a la cola y se elimina del mapa.

Se pide implementar el algoritmo de Kahn para obtener la ordenación topológica de un grafo.